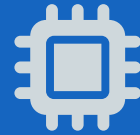


MODELAGEM COMPUTACIONAL



Expressões Regulares

Prof. Dr. José Jairo de S. e Silva

Aula 05

Expressões Regulares

OBJETIVOS DA AULA

🎯 Objetivos da Aula

- Entender a **definição formal** de expressão regular (ER)
- Conhecer as **3 operações regulares**: união, concatenação e estrela de Kleene
- Construir ERs para linguagens simples sobre $\{0, 1\}$
- Compreender a **precedência** dos operadores
- Relacionar **ERs, gramáticas regulares e autômatos finitos**
- Situar as linguagens regulares na **Hierarquia de Chomsky**
- Construir NFAs a partir de expressões regulares
- Exercícios

1. MOTIVAÇÃO

💡 Onde já usamos expressões regulares?

Expressões regulares (regex) aparecem em **muito mais lugares** do que parece:

Onde	O que faz	Exemplo
VS Code / Sublime	Buscar e substituir com padrões	Ctrl+H → func\w+\(
Terminal (grep)	Filtrar linhas em arquivos	grep -E '^Error' log.txt
Python / JavaScript	Validar entrada do usuário	re.match(r'\d{3}\.\d{3}', cpf)
HTML5	Validar campos de formulário	<input pattern="[0-9]{5}-[0-9]{3}">
SQL	Filtrar registros por padrão	WHERE nome REGEXP '^[A-M]'
Compiladores	Análise léxica (tokens)	identificadores, números, operadores

Observação: os exemplos desta seção usam **regex práticas** de ferramentas e linguagens de programação. A **definição formal** de expressão regular, na teoria da computação, é mais enxuta e será apresentada a seguir.

💡 Um exemplo concreto: validar e-mail

Para encontrar ou validar e-mails em um texto, usamos algo como:

`[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}`

- `[a-zA-Z0-9._%+-]+` → um ou mais caracteres válidos antes do @
- `@` → o símbolo @ literal
- `[a-zA-Z0-9.-]+` → o domínio (ex: `gmail`, `up.edu`)
- `\.` → o ponto literal (separador)
- `[a-zA-Z]{2,}` → extensão com 2+ letras (`br`, `com`, `edu`)

Mas atenção: essa já é uma **regex prática**, com classes de caracteres, quantificadores e escapes. Nesta aula, vamos ver a **versão formal** da teoria e depois conectar essa base às regex usadas em programação.

</> ERs formais vs regex em programação

A notação formal difere um pouco da notação usada em programação:

Formal	Programação	Significado
$R_1 \cup R_2$	R1 R2	União (alternativa)
$R_1 \circ R_2$	R1R2	Concatenação
R^*	R*	Zero ou mais
R^+	R+	Um ou mais
<i>um símbolo de Σ</i>	.	Qualquer símbolo do alfabeto
Σ^*	.*	Qualquer string

As regex de linguagens de programação usam a mesma base, mas acrescentam **extensões** de conveniência.

Observação: a tabela mostra **correspondências intuitivas** entre a notação formal e a notação de regex prática. Em engines reais, alguns detalhes podem variar.

💡 Três formas vistas até aqui

Nas aulas anteriores, vimos duas formas de descrever linguagens regulares. Nesta aula, entra a terceira:

1. DFA (Aula 3)

Máquina determinística com estados e transições. Reconhece a linguagem.

2. NFA (Aula 4)

Máquina não-determinística. Mesmo poder que DFA, mas mais concisa.

3. Expressão Regular (esta aula!)

Notação **algébrica** que descreve a linguagem usando operações sobre strings.

Até aqui, as três formas são **equivalentes**: descrevem exatamente as **linguagens regulares**. Mais adiante veremos uma **quarta forma equivalente**: gramáticas regulares.

Relembrando: as 3 operações regulares

Da Aula 2, já conhecemos as operações sobre linguagens:

1. **União:** $A \cup B = \{w \mid w \in A \text{ ou } w \in B\}$

2. **Concatenação:** $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$

3. **Estrela de Kleene:** $A^* = \{x_1x_2 \cdots x_k \mid k \geq 0, x_i \in A\}$

A estrela de Kleene também é chamada de **fecho de Kleene** (*Kleene closure*): são sinônimos. Variante: $A^+ = A \circ A^*$ é o **fecho positivo** (*positive closure*) → **uma** ou mais repetições (exclui ε se $\varepsilon \notin A$).

Exemplos com $A = \{0\}$ e $B = \{1\}$:

- $A \cup B = \{0, 1\}$
- $A \circ B = \{01\}$
- $A^* = \{\varepsilon, 0, 00, 000, \dots\}$

As expressões regulares são construídas **exatamente** a partir dessas 3 operações. Elas formam a **notação algébrica formal** usada para descrever linguagens regulares.

2. DEFINIÇÃO FORMAL DE EXPRESSÃO REGULAR

Definição 5.1: Expressão Regular

Dizemos que R é uma **expressão regular** (ER) sobre o alfabeto Σ se R é:

1. a , para algum $a \in \Sigma$ (um símbolo do alfabeto)
2. ε (a string vazia)
3. $\emptyset$ (a linguagem vazia)
4. $(R_1 \cup R_2)$, onde R_1 e R_2 são ERs (ex: $0 \cup 1$)
5. $(R_1 \circ R_2)$, onde R_1 e R_2 são ERs (ex: $0 \circ 1$ ou simplesmente 01)
6. (R_1^*) , onde R_1 é uma ER (ex: 0^*)

Interpretação: os itens 1-3 são os **casos base** (ERs atômicas). Os itens 4-6 são os **casos recursivos** (combinam ERs menores). Toda ER é construída aplicando essas regras um número finito de vezes. A seguir, veremos qual **linguagem** cada ER descreve.

☰ Linguagem descrita por uma ER

Cada ER R descreve uma linguagem $L(R)$:

ER R	Linguagem $L(R)$
a	$\{a\}$ (a linguagem com uma única string a)
ε	$\{\varepsilon\}$ (a linguagem contendo só a string vazia)
$\emptyset$	$\emptyset$ (a linguagem sem nenhuma string)
$R_1 \cup R_2$	$L(R_1) \cup L(R_2)$
$R_1 \circ R_2$	$L(R_1) \circ L(R_2)$
R_1^*	$(L(R_1))^*$

Notações simplificadas: escrevemos $R_1 R_2$ em vez de $R_1 \circ R_2$ (omitimos o operador de concatenação, como na multiplicação em álgebra). Também usamos Σ para representar $(a_1 \cup a_2 \cup \dots \cup a_k)$, onde a_i são todos os símbolos do alfabeto. Portanto, Σ^* = "qualquer string sobre o alfabeto".

↑ Precedência dos operadores

Assim como na aritmética ($\times$ antes de $+$), as operações de ER têm **precedência**:

Prioridade	Operação	Símbolo
1 (mais alta)	Estrela de Kleene	*
2	Concatenação	o (implícita)
3 (mais baixa)	União	$\cup$

Exemplo: a ER $01^* \cup 1$ é interpretada como:

$$(0 \circ (1^*)) \cup 1$$

e **não** como $(01)^* \cup 1$ nem como $0(1^* \cup 1)$.

São elementos de $01^* \cup 1$: $1, 0, 01, 011, 0111, 01111, \dots$

Dica: na dúvida, use parênteses! Mas a precedência permite simplificar a notação.

Notações alternativas

Na literatura, existem **duas notações** comuns para a operação de união em ERs:

Este curso (Sipser)	Outra notação comum	Significado
$R_1 \cup R_2$	$R_1 + R_2$	União
$R_1 \circ R_2$ ou $R_1 R_2$	$R_1 R_2$	Concatenação
R^*	R^*	Estrela de Kleene

Exemplo: a ER $(01)^* \cup (10)^*$ pode aparecer escrita como $(01)^* + (10)^*$ em outros livros.

Neste curso, usaremos preferencialmente $\cup$, mas saiba reconhecer ambas as notações. Em exercícios e provas, qualquer uma das duas é aceita.

⚠ Cuidado: ε vs $\emptyset$

Uma confusão muito comum:

ε (string vazia)

- É uma **string** (de comprimento 0)
- $L(\varepsilon) = \{\varepsilon\}$
- A linguagem contém **uma** string
- ε concatenado com qualquer string w dá w

$\emptyset$ (linguagem vazia)

- É uma **linguagem** (sem elementos)
- $L(\emptyset) = \emptyset$
- A linguagem contém **zero** strings
- $\emptyset$ concatenado com qualquer linguagem dá $\emptyset$

Resumo: ε é uma **string** (existe, mas tem comprimento 0). $\emptyset$ é uma **linguagem** (não contém string alguma). São objetos de tipos diferentes!

⚠ Consequências de ε vs $\emptyset$

1. Concatenação com ε (identidade):

$R \circ \varepsilon = R$. Concatenar qualquer string com a string vazia não muda nada. Ex: $01 \circ \varepsilon = 01$.

2. Concatenação com $\emptyset$ (absorvente):

$R \circ \emptyset = \emptyset$. Para formar uma string de $R \circ \emptyset$, precisaríamos escolher uma string de R e uma de $\emptyset$. Como $\emptyset$ não tem nenhuma string, o resultado é vazio. Ex: $01 \circ \emptyset = \emptyset$.

3. Estrela de ε :

$\varepsilon^* = \{\varepsilon\}$. Repetir a string vazia qualquer número de vezes sempre produz ε .

4. Estrela de $\emptyset$:

$\emptyset^* = \{\varepsilon\}$. Parece estranho! Mas lembre: $A^* = \{x_1 \dots x_k \mid k \geq 0\}$. Quando $k = 0$, a concatenação de **zero** strings é ε , independente de A . Mesmo sem nenhuma string para escolher, zero repetições produz ε .

☰ Identidades úteis de ERs

Algumas identidades que ajudam a simplificar expressões regulares:

Identidade	Explicação
$R \cup \emptyset = R$	$\emptyset$ é identidade da união
$R \circ \varepsilon = R$	ε é identidade da concatenação
$R \circ \emptyset = \emptyset$	$\emptyset$ absorve na concatenação
$R \cup R = R$	União é idempotente
$\emptyset^* \equiv \varepsilon$	ER equivalente à linguagem $\{\varepsilon\}$
$(R^*)^* = R^*$	Estrela dupla
$R^+ = R \circ R^*$	Uma ou mais repetições

Abreviação comum: $R^+ = RR^*$ significa "uma ou mais repetições de R ". Note que $R^* = R^+ \cup \varepsilon$.

3. EXEMPLOS DE EXPRESSÕES REGULARES

 Exemplo 5.1: strings com exatamente um 1

Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ contém exatamente um } 1\}$

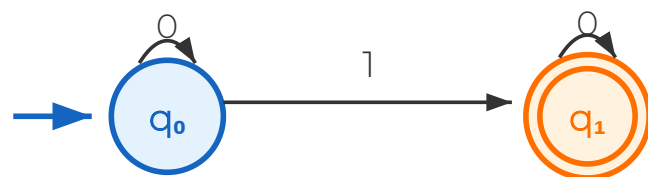
ER: 0^*10^*

Leitura: "zero ou mais 0s, seguido de um 1, seguido de zero ou mais 0s."

Strings aceitas: 1, 01, 10, 010, 0010, 100, 00100, ...

Strings rejeitadas: ϵ , 0, 11, 101, 0110, ...

NFA da aula 4 para esta linguagem:



Exemplo 5.2: strings que contêm 001

Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ contém a substring } 001\}$

ER: $\Sigma^*001\Sigma^*$

onde $\Sigma = (0 \cup 1)$, portanto:

$$\Sigma^*001\Sigma^* = (0 \cup 1)^*001(0 \cup 1)^*$$

Leitura: "qualquer coisa, seguido de 001, seguido de qualquer coisa."

Strings aceitas: 001, 0010, 1001, 00100, 110011, ...

Strings rejeitadas: ϵ , 0, 01, 010, 111, 0101, ...

 **No mundo real:** buscar uma substring dentro de um texto é a ideia por trás do **grep**. O comando **grep -E '001' arquivo.txt** segue a mesma lógica: encontrar linhas onde aparece o padrão **001** em qualquer posição.

 Exemplo 5.3: strings que terminam em 01

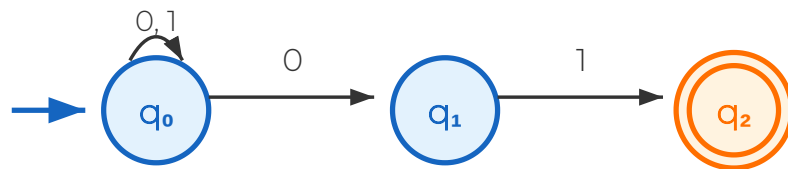
Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ termina em } 01\}$

ER: $(0 \cup 1)^*01$

Leitura: "qualquer coisa, seguido de 01 no final."

Strings aceitas: 01, 001, 101, 0101, 1101, ...

Conexão com a Aula 4: este é exatamente o NFA que usamos como exemplo principal!



Exemplo 5.4: strings de comprimento par

Linguagem: $L = \{w \in \{0, 1\}^* \mid |w| \text{ é par}\}$

ER: $(\Sigma\Sigma)^* = ((0 \cup 1)(0 \cup 1))^*$

Leitura: "zero ou mais repetições de pares de símbolos."

Strings aceitas: $\varepsilon, 00, 01, 10, 11, 0000, 0101, \dots$

Strings rejeitadas: $0, 1, 010, 111, 00100, \dots$

 **No mundo real:** esse padrão modela linguagens de **comprimento par**. Ele aparece em verificações simples de paridade/comprimento e pode ser reconhecido por um autômato com 2 estados.

Exemplo 5.5: strings sem dois 0s consecutivos

Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ não contém } 00\}$

ER: $1^*(01^+)^*(0 \cup \varepsilon)$

Raciocínio: podemos dividir qualquer string sem "00" em blocos:

- Começa com zero ou mais 1s: 1^*
- Seguido de blocos "0 seguido de um ou mais 1s": $(01^+)^*$
- Pode terminar com um 0 opcional: $(0 \cup \varepsilon)$

Strings aceitas: $\varepsilon, 0, 1, 01, 10, 101, 010, 1010, \dots$

Strings rejeitadas: $00, 100, 001, 0010, \dots$

 **No mundo real:** essa linguagem modela **restrições locais** em que o padrão **00** é proibido. Esse tipo de restrição aparece em esquemas de codificação, protocolos e formatos que vetam certas sequências.

 Exemplo 5.6: strings com 0s e 1s alternados

Linguagem: $L = \{w \in \{0, 1\}^* \mid w \text{ tem 0s e 1s alternados}\}$

$L = \{\epsilon, 0, 1, 01, 10, 010, 101, 0101, 1010, \dots\}$

ER: $(01)^* \cup (10)^* \cup 0(10)^* \cup 1(01)^*$

Raciocínio: toda string alternada se encaixa em exatamente um destes 4 casos:

Caso	Gera	Exemplos
$(01)^*$	começa com 0, termina com 1	$\epsilon, 01, 0101, 010101$
$(10)^*$	começa com 1, termina com 0	$\epsilon, 10, 1010, 101010$
$0(10)^*$	começa com 0, termina com 0	$0, 010, 01010$
$1(01)^*$	começa com 1, termina com 1	$1, 101, 10101$

A união cobre todas as strings alternadas possíveis.

Exemplo 5.6: justificativa passo a passo

Vamos verificar que $(01)^* \cup (10)^* \cup 0(10)^* \cup 1(01)^*$ é uma ER válida, seguindo a definição:

Passo 1: 0 e 1 são ERs válidas (caso base, $a \in \Sigma$). $L(0) = \{0\}$, $L(1) = \{1\}$.

Passo 2: 01 é ER válida (concatenação). $L(01) = \{01\}$.

Passo 3: $(01)^*$ é ER válida (estrela). $L((01)^*) = \{\epsilon, 01, 0101, \dots\}$.

Passo 4: $(10)^*$ é ER válida (mesma lógica). $L((10)^*) = \{\epsilon, 10, 1010, \dots\}$.

Passo 5: $0(10)^*$ é ER válida (concatenação). $L(0(10)^*) = \{0, 010, 01010, \dots\}$.

Passo 6: $1(01)^*$ é ER válida (concatenação). $L(1(01)^*) = \{1, 101, 10101, \dots\}$.

Passo 7: Aplicando a união 3 vezes, obtemos a ER completa, que descreve exatamente L .

Este tipo de justificativa passo a passo mostra como a definição indutiva **constrói** ERs complexas a partir de ERs simples.

Exemplo 5.7: ERs com outros alfabetos

ERs funcionam sobre **qualquer** alfabeto finito, não apenas $\{0, 1\}$.

a) $\Sigma = \{a, b, c\}$. A ER $a(a \cup b \cup c)^*c$ descreve:

Strings que **começam com a e terminam com c**. Ex: *ac, abc, aabbcc, acbc*.

b) $\Sigma = \{A, C, G, T\}$ (DNA). A ER $(A \cup T)(A \cup T \cup C \cup G)^*(A \cup T)$ descreve:

Sequências de DNA que **começam e terminam** com uma base fraca (*A* ou *T*).

c) $\Sigma = \{+, -, 0, 1, \dots, 9\}$. A ER $(+ \cup - \cup \epsilon)(0 \cup (1 \cup \dots \cup 9)(0 \cup \dots \cup 9)^*)$ descreve:

Números inteiros com sinal opcional, incluindo o **0**, e sem zeros à esquerda nos demais casos.
Ex: **0, 42, -7, +130**.

Exemplo 5.8: identificadores em linguagens de programação

$$\Sigma = \{a, \dots, z, A, \dots, Z, 0, \dots, 9, _ \}$$

Seja $\ell = (a \cup \dots \cup z \cup A \cup \dots \cup Z)$ e $d = (0 \cup \dots \cup 9)$.

ER: $(\ell \cup _)(\ell \cup d \cup _)^*$

Seja essa a linguagem: strings que **começam com letra ou _**, seguidas de zero ou mais letras, dígitos ou _.

Exemplos aceitos: x , $count$, $_temp$, $myVar2$

Exemplos não aceitos: $2nome$, $a-b$

 Exemplo 5.8: por que *_temp* é aceito?

Vamos verificar passo a passo na ER $(\ell \cup _)(\ell \cup d \cup _)^*$:

Posição	Símbolo	Parte da ER	Aceito?
1º	<u> </u>	$(\ell \cup _)$ exige letra ou <u> </u>	✓
2º	t	$(\ell \cup d \cup _)^*$ aceita letra	✓
3º	e	idem	✓
4º	m	idem	✓
5º	p	idem	✓

O primeiro símbolo casa com $(\ell \cup _)$, e os demais casam com $(\ell \cup d \cup _)^*$.

E por que *2nome* é rejeitado? O primeiro símbolo é 2 (dígito). A ER exige que o primeiro símbolo case com $(\ell \cup _)$, que só aceita **letra ou**. Dígito não está nesse conjunto, então a string é rejeitada logo no início.

Exemplo 5.8: da teoria à prática

A ER formal $(\ell \cup _)(\ell \cup d \cup _)^*$ tem uma tradução direta para regex de programação:

ER formal	Regex	Significado
$\ell = (a \cup \dots \cup z \cup A \cup \dots \cup Z)$	<code>[a-zA-Z]</code>	qualquer letra
$d = (0 \cup \dots \cup 9)$	<code>[0-9]</code>	qualquer dígito
$(\ell \cup _)$	<code>[a-zA-Z_]</code>	primeiro caractere
$(\ell \cup d \cup _)^*$	<code>[a-zA-Z0-9_]*</code>	restante (zero ou mais)

Regex completa: `[a-zA-Z_][a-zA-Z0-9_]*`

Isso é exatamente a regra de identificadores em linguagens como C, Java e Python. A teoria que estamos estudando é a base formal dessas regras!

🔗 Pratique

Qual linguagem é descrita?

1. $0(0 \cup 1)^*0$ 2. $(\varepsilon \cup 0)1^*$ 3. $(01^+)^*$

Escreva uma ER para...

4. Strings sobre $\{0, 1\}$ que começam com 0.

5. Strings sobre $\{0, 1\}$ com no máximo dois 1s.

6. Strings sobre $\{0, 1\}$ onde todo 0 é seguido de pelo menos um 1.

☑ Respostas: qual linguagem é descrita?

1. $0(0 \cup 1)^*0$ → Strings que **começam e terminam com 0** (comprimento ≥ 2).
2. $(\varepsilon \cup 0)1^*$ → ε ou um 0 opcional seguido de zero ou mais 1s: $\{\varepsilon, 1, 11, 0, 01, 011, \dots\}$
3. $(01^+)^*$ → ε ou blocos "0 seguido de um ou mais 1s": $\{\varepsilon, 01, 011, 0101, 01011, \dots\}$

☑ Respostas: escreva uma ER para...

4. Strings sobre $\{0, 1\}$ que começam com 0.

$$0(0 \cup 1)^* = 0\Sigma^*$$

5. Strings sobre $\{0, 1\}$ com no máximo dois 1s.

$$0^*(1 \cup \varepsilon) 0^*(1 \cup \varepsilon) 0^*$$

6. Strings sobre $\{0, 1\}$ onde todo 0 é seguido de pelo menos um 1.

$$1^*(01^+)^*$$

4. EQUIVALÊNCIA COM AUTÔMATOS FINITOS

↔ Teorema de Kleene: ER ↔ Linguagens Regulares

O resultado central desta aula:

Teorema 5.1 (Kleene): Uma linguagem é regular **se e somente se** pode ser descrita por uma expressão regular.

Isso significa que temos **três formas equivalentes** de definir linguagens regulares:

$$\boxed{\text{DFA} \leftrightarrow \text{NFA} \leftrightarrow \text{ER}}$$

Para provar, precisamos mostrar duas direções:

1. **ER $\Rightarrow$ NFA**: toda ER pode ser convertida em um NFA (e portanto em um DFA)
2. **DFA $\Rightarrow$ ER**: todo DFA pode ser convertido em uma ER

Nesta aula, provaremos apenas a direção **ER $\Rightarrow$ NFA** (toda ER vira um NFA). A direção inversa (**DFA $\Rightarrow$ ER**) será vista em aula futura.

Estratégia da prova: construção indutiva

A prova funciona por **indução na estrutura** da ER. A ideia é simples:

1. Para cada **caso base** (peça elementar), construímos um NFA pequeno
2. Para cada **operação** (forma de combinar), mostramos como juntar NFAs

Casos base (3 tipos de ER atômica):

ER	Linguagem	NFA
a (símbolo)	$\{a\}$	2 estados, 1 transição
ϵ	$\{\epsilon\}$	1 estado (inicial = final)
$\emptyset$	$\emptyset$	1 estado (inicial, sem final)

Estratégia da prova: operações recursivas

Operações recursivas (3 formas de combinar ERs menores):

Operação	ER	Como combinar NFAs
União	$R_1 \cup R_2$	Novo início escolhe entre N_1 e N_2
Concatenação	$R_1 \circ R_2$	Saída de N_1 conecta à entrada de N_2
Estrela	R_1^*	Repetir N_1 zero ou mais vezes

Visão geral: qualquer ER, por mais complexa que seja, é construída combinando esses 6 blocos. A construção do NFA segue a mesma estrutura recursiva:

1. Decompor a ER nas sub-expressões
2. Construir o NFA de cada sub-expressão (começando pelos casos base)
3. Combinar os NFAs usando a operação correspondente (união, concatenação ou estrela)

Para "colar" os NFAs nas operações recursivas, usamos uma ferramenta auxiliar: **transições- ϵ** .

⚙ Ferramenta auxiliar: transições- ϵ

Para conectar os NFAs nas construções, usamos **transições- ϵ** (transições sem consumir símbolo).

Uma transição- ϵ permite que o NFA mude de estado **sem ler nenhum símbolo** da entrada.



Significado: estando em q_0 , o NFA pode ir para q_1 **gratuitamente**, sem consumir nenhum símbolo da entrada.

Analogia: imagine que você está num aeroporto (q_0). Uma transição- ϵ é como uma **passarela** para outro terminal (q_1): você pode ir até lá sem precisar de passagem (sem "gastar" um símbolo). Você escolhe se vai ou não.

⚙️ Transições- ϵ : por que usar?

Para que servem? Servem para "colar" dois NFAs sem precisar redesenhar estados ou transições. Ao invés de modificar a estrutura interna dos autômatos, simplesmente adicionamos uma passagem gratuita entre eles.

Transições- ϵ aumentam o poder computacional?

Não! Reconhecem as **mesmas linguagens**. Toda transição- ϵ pode ser eliminada:

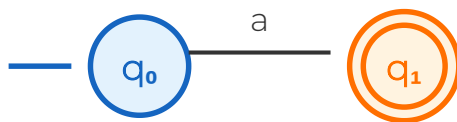
1. Para cada estado q , calcule o **fecho- ϵ** : todos os estados alcançáveis a partir de q seguindo apenas transições- ϵ
2. Redirecione as transições para incluir esses estados
3. O NFA resultante, sem ϵ , reconhece a mesma linguagem

Transições- ϵ são uma **conveniência notacional** que simplifica as construções a seguir. Não adicionam poder, apenas tornam as construções mais fáceis de descrever.

⚙️ Caso base 1: $ER = a$ (um símbolo do alfabeto)

A construção é **indutiva**, seguindo a estrutura recursiva da ER. Começamos pelos 3 casos base.

Caso 1: $ER = a$

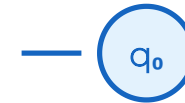


- O NFA tem **2 estados**: q_0 (inicial) e q_1 (final, círculo duplo)
- **Única transição**: ao ler o símbolo a , vai de q_0 para q_1
- Aceita **exatamente** a string a . Qualquer outra string é rejeitada
- Linguagem reconhecida: $L = \{a\}$

⚙️ Casos base 2 e 3: $ER = \varepsilon$ e $ER = \emptyset$

Caso 2: $ER = \varepsilon$

Caso 3: $ER = \emptyset$



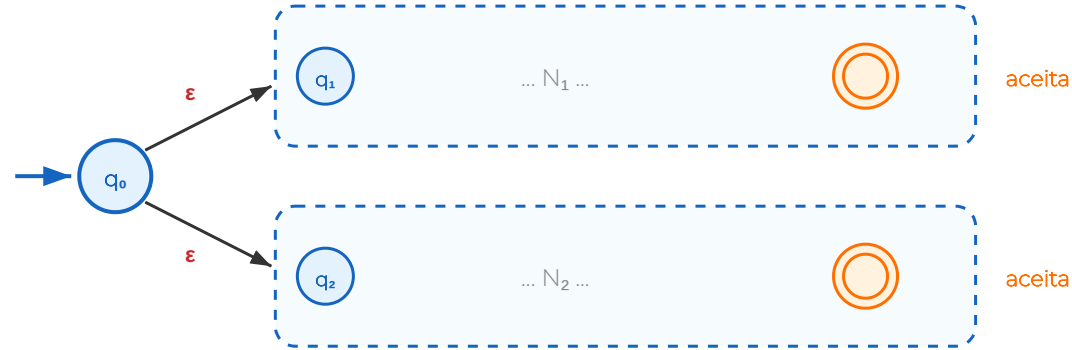
Caso 2 (ε): o NFA tem **1 estado** que é inicial **e** final (círculo duplo). Como $q_0 \in F$, a string vazia é aceita imediatamente, sem ler nada. $L(\varepsilon) = \{\varepsilon\}$.

Caso 3 ($\emptyset$): o NFA tem **1 estado** que é inicial mas **não** é final (círculo simples). Como $q_0 \notin F$ e não há transições, nenhuma string é aceita. $L(\emptyset) = \emptyset$.

Compare os diagramas: a única diferença é o círculo duplo (estado final) vs círculo simples. Essa pequena diferença muda tudo: um aceita ε , o outro não aceita nada.

⚙️ Operação recursiva 1: União ($R_1 \cup R_2$)

Dados os NFAs N_1 (para R_1) e N_2 (para R_2), construímos o NFA para $R_1 \cup R_2$:



Ideia: criar um novo estado inicial q_0 com transições- ϵ para os inícios de N_1 e N_2 . O NFA "escolhe" qual caminho seguir.

⚙️ União: passo a passo

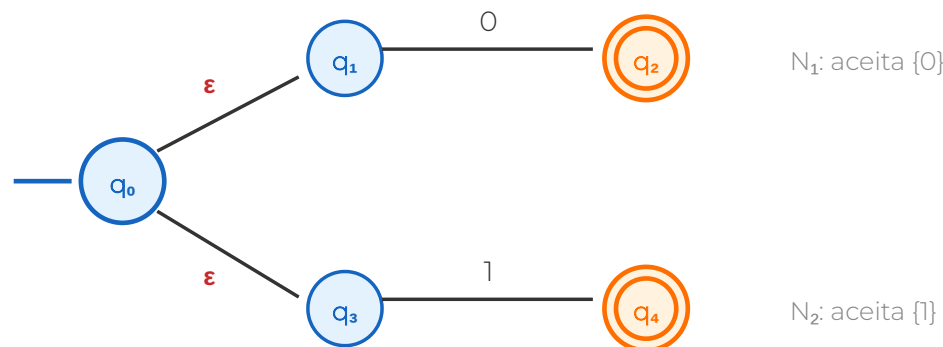
Passo a passo da construção:

1. Criar um **novo estado inicial** q_0 (que não pertence a N_1 nem a N_2)
2. Adicionar $q_0 \xrightarrow{\varepsilon} q_1$ (início de N_1) e $q_0 \xrightarrow{\varepsilon} q_2$ (início de N_2)
3. Os estados finais de N_1 e N_2 **continuam sendo finais**

Por que funciona? Ao receber uma string w , o NFA segue por ε para N_1 **ou** N_2 . Se $w \in L(R_1)$, o caminho por N_1 aceita. Se $w \in L(R_2)$, o caminho por N_2 aceita. Basta **um** caminho aceitar.

⚙️ União: exemplo concreto

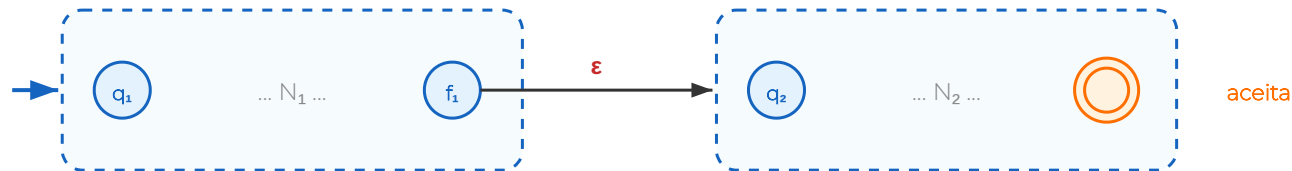
Exemplo: N_1 aceita $\{0\}$ e N_2 aceita $\{1\}$. O NFA da união aceita $\{0, 1\}$:



String	Caminho no NFA	Resultado
$w = 0$	$q_0 \xrightarrow{\varepsilon} q_1 \xrightarrow{0} q_2$ (final)	✓
$w = 1$	$q_0 \xrightarrow{\varepsilon} q_3 \xrightarrow{1} q_4$ (final)	✓
$w = 01$	nenhum caminho chega a um estado final	✗

⚙️ Operação recursiva 2: Concatenação ($R_1 \circ R_2$)

Dados os NFAs N_1 e N_2 , construímos o NFA para $R_1 \circ R_2$:



Ideia: os estados finais de N_1 deixam de ser finais e ganham transições- ϵ para o início de N_2 . Quando N_1 "termina" de processar a primeira parte da string, automaticamente começa N_2 para processar o restante.

⚙️ Concatenação: passo a passo

Passo a passo da construção:

1. O estado inicial é o de N_1 (não muda)
2. Os estados finais de N_1 **deixam de ser finais**
3. Cada antigo estado final f_1 de N_1 ganha $f_1 \xrightarrow{\varepsilon} q_2$ (início de N_2)
4. Os estados finais de N_2 são os **únicos finais** do novo NFA

Por que funciona? O NFA primeiro processa a string por N_1 . Quando N_1 chega a um antigo estado final (f_1), a transição- ε passa o controle para N_2 , que processa o restante. A string toda é aceita somente se a primeira parte pertence a $L(R_1)$ e a segunda parte pertence a $L(R_2)$.

⚙️ Concatenação: exemplo concreto

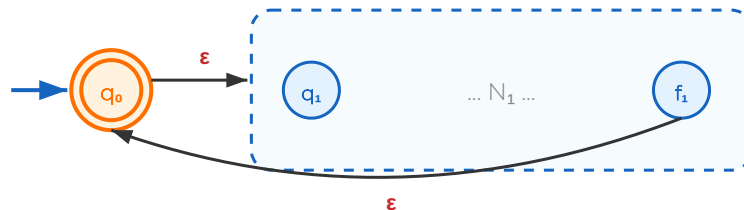
Exemplo: N_1 aceita $\{0\}$ e N_2 aceita $\{1\}$. O NFA da concatenação aceita $\{01\}$:



String	Caminho no NFA	Resultado
$w = 01$	$q_1 \xrightarrow{0} q_2 \xrightarrow{\varepsilon} q_3 \xrightarrow{1} q_4$ (final)	✓
$w = 0$	$q_1 \xrightarrow{0} q_2 \xrightarrow{\varepsilon} q_3$, mas q_3 não é final	✗
$w = 10$	q_1 não tem transição com 1 , fica preso	✗

⚙️ Operação recursiva 3: Estrela de Kleene (R_1^*)

Dado o NFA N_1 , construímos o NFA para R_1^* :



Ideia: repetir N_1 zero ou mais vezes. O novo q_0 é final (aceita ϵ , que corresponde a zero repetições) e uma transição- ϵ de volta permite repetir N_1 .

⚙️ Estrela de Kleene: passo a passo

Passo a passo da construção:

1. Criar novo estado inicial q_0 , que **também é final** (aceita ε , zero repetições)
2. Adicionar $q_0 \xrightarrow{\varepsilon} q_1$ (início de N_1)
3. Cada estado final f_1 de N_1 ganha $f_1 \xrightarrow{\varepsilon} q_0$ (volta ao início para repetir)
4. Os antigos estados finais de N_1 **deixam de ser finais**. Só q_0 é final

Por que f_1 deixa de ser final? Se f_1 continuasse final, o NFA poderia aceitar prefixos indesejados (parar no meio de uma repetição). A aceitação sempre termina em q_0 , garantindo que apenas concatenações **completas** de strings de $L(R_1)$ são aceitas.

Em resumo, o ciclo é: $q_0 \xrightarrow{\varepsilon} q_1 \rightarrow \dots \rightarrow f_1 \xrightarrow{\varepsilon} q_0 \xrightarrow{\varepsilon} q_1 \rightarrow \dots \rightarrow f_1 \xrightarrow{\varepsilon} q_0$. Cada volta completa consome uma string de $L(R_1)$.

Estrela de Kleene: exemplo concreto

Exemplo: se N_1 reconhece $\{ab\}$, o NFA para $(ab)^*$ aceita:

String	Caminho	Repetições	Resultado
ϵ	fica em q_0 (final)	0	✓
ab	$q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{a} \dots \xrightarrow{b} f_1 \xrightarrow{\epsilon} q_0$	1	✓
$abab$	$q_0 \xrightarrow{\epsilon} \dots \xrightarrow{ab} f_1 \xrightarrow{\epsilon} q_0 \xrightarrow{\epsilon} \dots \xrightarrow{ab} f_1 \xrightarrow{\epsilon} q_0$	2	✓

E as rejeitadas?

String	Por que é rejeitada?
a	entra em N_1 mas não chega a f_1 (falta o b)
ba	N_1 espera a primeiro, não tem transição com b
aba	primeira repetição ok (ab), mas sobra a que não completa

☒ Resumo das 3 construções

Operação	O que faz	Como conecta
União	Escolhe entre N_1 e N_2	Novo q_0 com ε para ambos
Concatenação	Encadeia N_1 depois N_2	Finais de N_1 com ε para início de N_2
Estrela	Repete N_1 zero ou mais vezes	Novo q_0 (final) com ε de ida e volta

Observações importantes:

- Na **união**, os estados finais de N_1 e N_2 continuam sendo finais
- Na **concatenação**, os estados finais de N_1 perdem o status de final
- Na **estrela**, os estados finais de N_1 perdem o status de final, e o novo q_0 é o único final
- Em todas as construções, usamos apenas transições- ε para conectar. A estrutura interna de cada NFA permanece intacta.

5. EXEMPLO COMPLETO: ER $\Rightarrow$ NFA

Exemplo 1: convertendo $(0 \cup 1)^*01$ em NFA

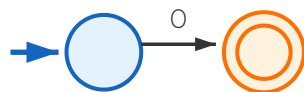
Vamos aplicar as construções para converter a ER $(0 \cup 1)^*01$ passo a passo.

Passo 1: identificar a estrutura. A ER é uma concatenação de 3 partes:

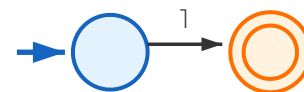
$$(0 \cup 1)^* \quad \circ \quad 0 \quad \circ \quad 1$$

Passo 2: construir os NFAs base para os símbolos **0** e **1**:

NFA para "0"



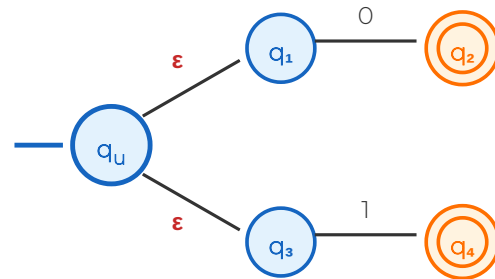
NFA para "1"



Precisamos de **duas cópias** do NFA para "0" e **duas** do NFA para "1" (uma para a união, outra para a concatenação final).

Exemplo 1: Passo 3, união $0 \cup 1$

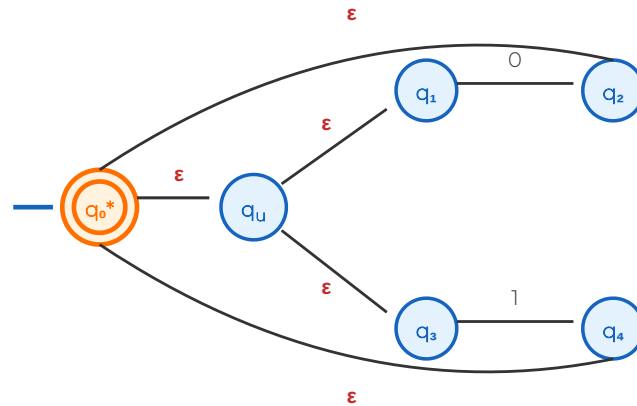
Passo 3: aplicar a construção de **união** aos NFAs de "0" e "1":



5 estados, aceit

 Exemplo 1: Passo 4, estrela $(0 \cup 1)^*$

Passo 4: aplicar a construção de **estrela** ao NFA da união:



q_2 e q_4 não são mais finais
Só q_0^* é final (aceita ϵ)

6 estados

Aceita $(0 \cup 1)^*$
= qualquer string
sobre $\{0,1\}$

O ciclo funciona assim: $q_0^* \xrightarrow{\epsilon} q_u \xrightarrow{\epsilon} (\text{ler } 0 \text{ ou } 1) \xrightarrow{\epsilon} q_0^*$ e pode repetir quantas vezes quiser.

Exemplo 1: Passos 5 e 6, concatenação final

Passo 5: concatenar $(0 \cup 1)^*$ com o NFA para "0":

- q_0^* **deixa de ser final** (pois estamos concatenando)
- Adicionar $q_0^* \xrightarrow{\varepsilon} q_5$ (início do NFA para "0")
- Novo estado q_6 (final do NFA para "0") é o estado final temporário
- Total parcial: **8 estados**

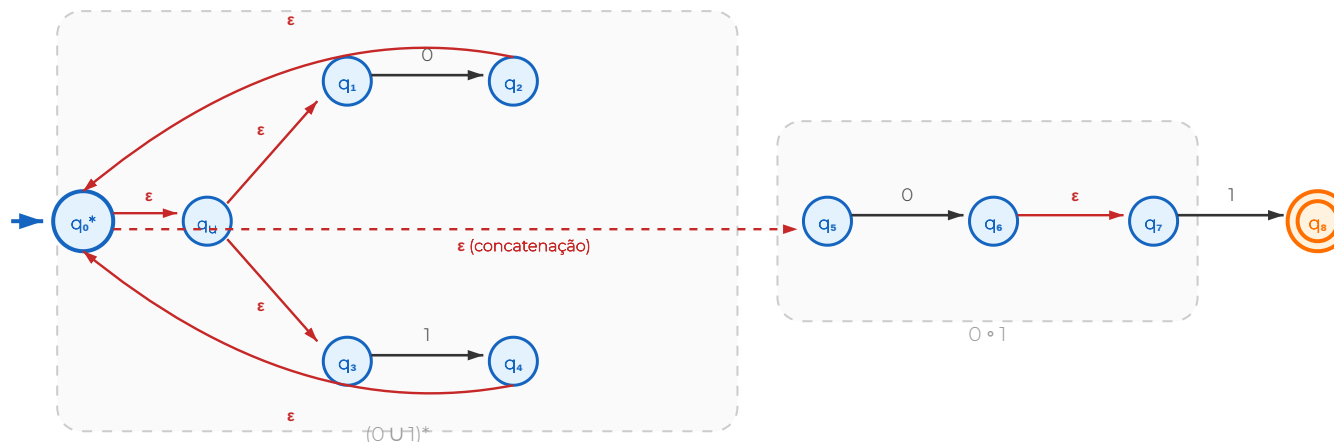
Passo 6: concatenar o resultado com o NFA para "1":

- q_6 **deixa de ser final**
- Adicionar $q_6 \xrightarrow{\varepsilon} q_7$ (início do NFA para "1")
- Novo estado q_8 é o **único estado final** de todo o NFA
- Total final: **10 estados**

Resultado: o NFA formal para $(0 \cup 1)^*01$ tem **10 estados** e muitas transições- ε .
Vamos ver o diagrama completo no próximo slide.

Exemplo 1: NFA formal completo

NFA formal para $(0 \cup 1)^*01$ com **10 estados** (7 transições- ϵ , 4 transições com símbolo):

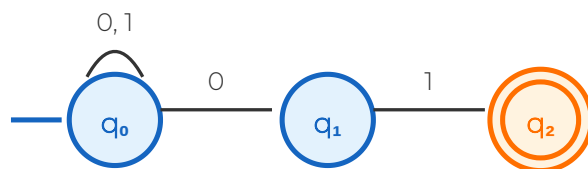


A parte esquerda (fundo cinza) é $(0 \cup 1)^*$. A parte direita é a concatenação com **01**. As setas vermelhas tracejadas são transições- ϵ .

Exemplo 1: formal vs direto

A construção formal gera um NFA correto, mas com **10 estados** e muitas transições- ϵ .

Na prática, podemos construir um NFA **direto** bem mais simples:



Apenas **3 estados**, sem transições- ϵ ! A intuição é clara: **q_0** lê qualquer coisa (loop), ao ler **0** vai para **q_1** , e ao ler **1** vai para **q_2** (aceita).

Construção formal	Construção direta
10 estados	3 estados
7 transições- ϵ	0 transições- ϵ
Correto, mas grande	Correto e compacto
Método mecânico (funciona sempre)	Requer intuição sobre a linguagem

Exemplo 2: convertendo 0^*10^* em NFA

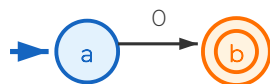
Vamos converter a ER 0^*10^* passo a passo. Esta ER descreve a linguagem das strings com **exatamente um 1**.

Passo 1: identificar a estrutura. A ER é uma concatenação de 3 partes:

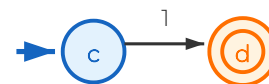
$$0^* \quad \circ \quad 1 \quad \circ \quad 0^*$$

Passo 2: construir os NFAs base:

NFA para "0"



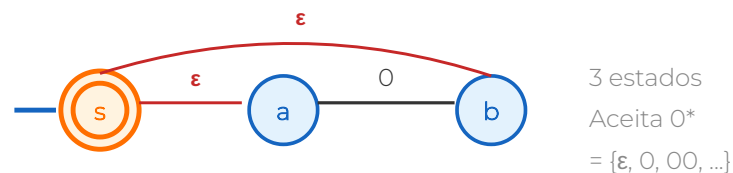
NFA para "1"



Usamos letras ($a, b, c, d, \dots$) para os estados, evitando confusão com o exemplo anterior.

✎ Exemplo 2: Passos 3 e 4, construindo 0^*

Passo 3: aplicar a **estrela** ao NFA para "0":



Passo 4: precisamos de **duas cópias** de 0^* (uma para o início e outra para o fim da concatenação). A segunda cópia usa estados s', a', b' .

Agora temos todas as peças: NFA para 0^* (estados s, a, b), NFA para 1 (estados c, d), e NFA para 0^* (estados s', a', b'). Total: **9 estados** antes das concatenações.

Exemplo 2: Passos 5 e 6, concatenação

Passo 5: concatenar 0^* com o NFA para "1":

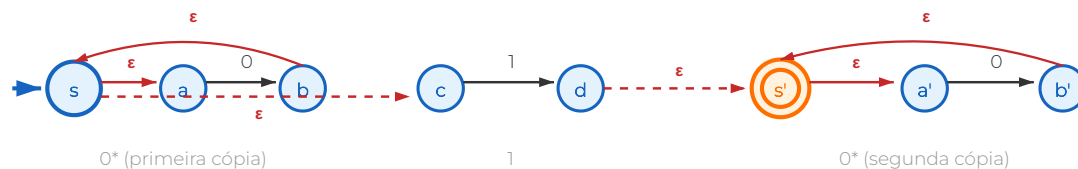
- s deixa de ser final
- Adicionar $s \xrightarrow{\varepsilon} c$ (início do NFA para "1")

Passo 6: concatenar o resultado com 0^* (segunda cópia):

- d deixa de ser final
- Adicionar $d \xrightarrow{\varepsilon} s'$ (início da segunda cópia de 0^*)

O NFA resultante tem **9 estados**. O único estado final é s' (início da segunda estrela).

Diagrama do NFA formal para 0^*10^* :



Exemplo 2: testando o NFA

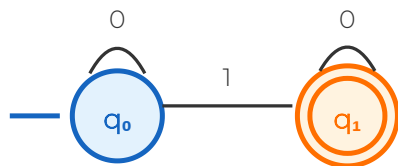
Vamos testar o NFA formal para 0^*10^* com algumas strings:

String	Caminho	Resultado
1	$s \xrightarrow{\varepsilon} c \xrightarrow{1} d \xrightarrow{\varepsilon} s'$ (final)	✓
010	$s \xrightarrow{\varepsilon} a \xrightarrow{0} b \xrightarrow{\varepsilon} s \xrightarrow{\varepsilon} c \xrightarrow{1} d \xrightarrow{\varepsilon} s' \xrightarrow{\varepsilon} a' \xrightarrow{0} b' \xrightarrow{\varepsilon} s'$	✓
0010	loop 0^* duas vezes, depois 1, depois 0 via segunda cópia	✓
String	Por que é rejeitada?	
ε	s não é final, e sem ler 1 não alcança s'	
00	não lê 1, nunca chega a d e portanto a s'	
011	lê 0, depois 1 (ok), mas o segundo 1 não tem transição em a'	

A linguagem $L(0^*10^*) = \{w \in \{0, 1\}^* \mid w \text{ contém exatamente um } 1\}$.

Exemplo 2: formal vs direto

Assim como no exemplo anterior, podemos construir um NFA **direto** mais simples:



Apenas **2 estados**! Em q_0 , ler **0** faz um loop (zero ou mais **0**s). Ler **1** vai para q_1 (final). Em q_1 , ler **0** faz loop (zero ou mais **0**s depois do **1**).

Construção formal	Construção direta
9 estados	2 estados
6 transições- ϵ	0 transições- ϵ
Método mecânico	Requer compreender a linguagem

Lição dos dois exemplos: a construção formal funciona **sempre**, mas gera NFAs grandes. A construção direta é compacta, mas exige entender o que a ER descreve. Ambas reconhecem a **mesma linguagem**.

6. GRAMÁTICAS REGULARES E HIERARQUIA DE CHOMSKY

Hierarquia de Chomsky: onde entram as ERs?

As linguagens regulares ocupam o nível **mais restrito** da hierarquia, mas também o mais operacional:

Tipo	Classe	Forma de descrição	Reconhecedor
3	Regular	ERs e gramáticas regulares	Autômato finito
2	Livre de contexto	GLCs	Autômato com pilha
1	Sensível ao contexto	Gramáticas sensíveis ao contexto	MT linearmente limitada
0	Irrestrita	Gramáticas irrestritas	Máquina de Turing

Inclusão: **Tipo 3** $\subset$ **Tipo 2** $\subset$ **Tipo 1** $\subset$ **Tipo 0**. Toda linguagem regular é livre de contexto, mas a recíproca é falsa.

Gramáticas regulares: a 4ª forma equivalente

Uma **gramática** é um conjunto de **regras de substituição** (produções). Uma gramática regular à direita usa produções da forma:

- $A \rightarrow aB$ (não-terminal gera terminal + não-terminal)
- $A \rightarrow a$ (não-terminal gera apenas terminal)
- Em muitas convenções, também $S \rightarrow \epsilon$

onde letras maiúsculas (S, A, B) são **não-terminais** (variáveis) e minúsculas (a, b) são **terminais** (símbolos do alfabeto).

Ideia central: em uma gramática regular, cada produção gera no máximo **um terminal** e, opcionalmente, **um próximo não-terminal**.

Gramáticas regulares: exemplo e equivalência

Exemplo de gramática:

$$\begin{aligned} S &\rightarrow aA \\ A &\rightarrow aA \mid b \end{aligned}$$

Derivação (indicada por $\Rightarrow$):

$$S \Rightarrow aA \Rightarrow aaA \Rightarrow aab$$

Linguagem gerada:

$$L(G) = \{a^n b \mid n \geq 1\}$$

ER equivalente:

$$aa^*b \text{ (ou } a^+b\text{)}$$

Também existe um DFA/NFA equivalente.

Para linguagens regulares, temos quatro descrições equivalentes: **ER** $\leftrightarrow$ **gramática regular** $\leftrightarrow$ **NFA** $\leftrightarrow$ **DFA**.

Compiladores: léxico vs sintático

No front-end de um compilador, as classes da hierarquia aparecem em etapas diferentes:

Etapa	Tipo	Ferramenta teórica	Função
Análise léxica	Tipo 3	ERs / gramáticas regulares / AFs	Transformar caracteres em tokens
Análise sintática	Tipo 2	GLCs / autômatos com pilha	Verificar a estrutura do programa

Exemplos:

- O léxico reconhece padrões como identificadores, números e operadores.
- O sintático verifica se estruturas como `if (x) y = z;` obedecem à gramática da linguagem.

7. APLICAÇÕES PRÁTICAS

ERs no mundo real: de onde vêm os padrões?

Cada ER formal que vimos corresponde a um **problema real** de reconhecimento de padrões:

Padrão (formal ou metanotacional)	Aplicação no mundo real	Exemplo
$\Sigma^* p \Sigma^*$	Busca de substring (grep, Ctrl+F)	<code>grep "erro" log.txt</code>
$(\ell U_)(\ell U d U_)^*$	Identificadores em compiladores	<code>myVar, _count, x2</code>
$(s U \epsilon) d d^*$	Números inteiros com sinal	<code>42, -7, +130</code>
$d^3.d^3.d^3-d^2$	Validação de CPF (formato)	<code>123.456.789-00</code>
$(\Sigma\Sigma)^*$	Verificação de comprimento par	Strings de comprimento par
$(A U T)\Sigma^*(A U T)$	Bioinformática (sequências DNA)	<code>ATCGA, TGCAT</code>

Observação: nas linhas com ℓ , d e s , usamos abreviações metanotacionais. Em uma ER formal estrita, essas classes precisariam ser explicitamente expandidas como uniões.

A teoria de ERs **fundamenta** essas ferramentas. Em muitos casos, padrões regulares podem ser implementados com base em autômatos finitos, enquanto as regex práticas acrescentam conveniências sintáticas sobre essa base teórica.

</> Extensões comuns em regex de programação

Estas notações **não** fazem parte da definição formal de ER, mas são atalhos usados em ferramentas como **grep**, Python e JavaScript:

Notação	Nome	Significado	Equivalente formal
^	Âncora de início	Início da string	Sem equivalente: ERs formais já casam a string inteira
\$	Âncora de fim	Fim da string	Idem: a ER formal casa a string inteira
[abc]	Classe de caracteres	Um dentre a, b ou c	$(a \cup b \cup c)$
[a-z]	Faixa de caracteres	Qualquer letra de a até z	$(a \cup b \cup \dots \cup z)$
?	Opcional	Zero ou uma ocorrência	$(R \cup \epsilon)$
{n,m}	Repetição limitada	De n a m ocorrências	$R \circ \dots \circ R$ (n vezes) $\circ (R \cup \epsilon) \circ \dots$ (m-n vezes)
\	Escape	Trata o próximo caractere como literal	Não necessário: ERs formais não têm metacaracteres

✉ Exemplo de decomposição de uma regex real

Considere a regex:

`[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}`

Leitura por partes:

- `[a-zA-Z0-9._%+-]+` representa o nome do usuário do e-mail.
- `@` é um símbolo literal.
- `[a-zA-Z0-9.-]+` representa o domínio.
- `\.` força o ponto literal antes da extensão.
- `[a-zA-Z]{2,}` exige pelo menos duas letras na parte final.

Cada `[...]` funciona como uma grande união, `+` representa repetição positiva, `\.` escapa o ponto e `{2,}` é uma repetição limitada. É um bom exemplo de regex prática construída sobre a base teórica das ERs.

</> Exemplo em Python

```
import re

# ER formal: (0 u 1)*01
# Regex Python equivalente:
# [01] = classe de caracteres → (0 u 1)
# * = zero ou mais → estrela de Kleene
# 01 = literais concatenados
# re.fullmatch() já casa a string inteira (não precisa de ^ e $)

padrao = r'[01]*01'

testes = ['01', '001', '101', '0101', '10', '00', '1']
for s in testes:
    resultado = 'aceita' if re.fullmatch(padrao, s) else 'rejeita'
    print(f'{s:6s} -> {resultado}')
```

```
01      -> aceita
001     -> aceita
101     -> aceita
0101    -> aceita
10      -> rejeita
00      -> rejeita
1       -> rejeita
```

| **re.fullmatch()** casa a string **inteira** (como uma ER formal), dispensando ^ e \$.

8. RESUMO E EXERCÍCIOS

✓ Resumo da Aula

- **Expressão regular (ER):** notação algébrica para descrever linguagens regulares.
- **Definição recursiva:** construída a partir de a , ε , $\emptyset$ usando $\cup$, $\circ$, $*$.
- **Precedência:** $*$ > concatenação > união.
- ε (string vazia) $\neq \emptyset$ (linguagem vazia): $L(\varepsilon) = \{\varepsilon\}$, $L(\emptyset) = \emptyset$.
- **Regex de programação:** usam a mesma base das ERs formais, mas incluem extensões sintáticas extras.
- **Hierarquia de Chomsky:** linguagens regulares são **Tipo 3**.
- **Equivalência:** ER $\leftrightarrow$ gramática regular $\leftrightarrow$ DFA $\leftrightarrow$ NFA.
- **Construção ER $\Rightarrow$ NFA:** indutiva, usando transições- ε para união, concatenação e estrela.
- **Aplicações:** regex, grep, análise léxica; análise sintática já usa **Tipo 2**.

❓ Verificação Rápida

1. A ER 01^* descreve a mesma linguagem que $(01)^*$?

Resposta: Não! $01^* = 0(1^*) = \{0, 01, 011, 0111, \dots\}$, mas $(01)^* = \{\epsilon, 01, 0101, 010101, \dots\}$.

2. Qual é $L(\emptyset^*)$?

Resposta: $L(\emptyset^*) = \{\epsilon\}$. A estrela de Kleene sempre inclui ϵ (zero repetições).

3. A ER Σ^* aceita a string vazia?

Resposta: Sim! $\Sigma^* = (0 \cup 1)^*$ inclui ϵ (zero repetições do alfabeto).

Exercícios

Exercício 5.1 Descreva em português a linguagem de cada ER sobre $\Sigma = \{0, 1\}$:

(a) $0(0 \cup 1)^*0$ (b) $(\varepsilon \cup 0)1^*$ (c) $(01^+)^*$

Exercício 5.2 Escreva uma ER para cada linguagem sobre $\Sigma = \{0, 1\}$:

(a) Strings que começam com 0 e terminam com 1 (b) Strings com no máximo dois 1s (c) Strings onde todo 0 é seguido de pelo menos um 1

Exercícios (cont.)

Exercício 5.3 Construa o NFA correspondente a ER $01 \cup 0^*$ usando a construção indutiva vista em aula. Desenhe o diagrama de estados resultante.

Exercício 5.4 Simplifique cada ER usando as identidades vistas em aula:

(a) $\emptyset^* \cup 1$ (b) $R \circ \varepsilon$ (c) $R \cup \emptyset$

Exercício 5.5 Verdadeiro ou falso? Justifique.

(a) $L(\emptyset^*) = \{\varepsilon\}$ (b) $(0^*1^*)^* = (0 \cup 1)^*$ (c) $\emptyset \circ R = R$

Exercícios (cont.)

Exercício 5.6 Seja $\Sigma = \{a, b, c\}$. Escreva uma ER para a linguagem das strings sobre Σ que **começam e terminam com a** e têm **pelo menos um b** .

Exercício 5.7 Seja $\Sigma = \{a, b\}$. Escreva uma ER para a linguagem das strings sobre Σ que têm **tamanho ímpar**.

Exercício 5.8 Justifique passo a passo, usando a Definição 5.1, que a ER $(01)^* \cup 1(01)^*$ é uma expressão regular válida. Identifique a linguagem que ela descreve.

SOLUÇÕES DOS EXERCÍCIOS

✔ Solução do Exercício 5.1

Descreva em português a linguagem de cada ER:

(a) $0(0 \cup 1)^*0$

Strings que **começam com 0** e **terminam com 0**, com comprimento ≥ 2 . O meio pode ser qualquer combinação de 0s e 1s.

Exemplos: 00, 010, 000, 0110, 01010

(b) $(\epsilon \cup 0)1^*$

A string vazia, ou strings formadas por um **0 opcional** seguido de **zero ou mais 1s**.

Exemplos: ϵ , 1, 11, 0, 01, 011, 0111

(c) $(01^+)^*$

A string vazia, ou strings formadas por **blocos** onde cada bloco é um 0 seguido de **um ou mais 1s**.

Exemplos: ϵ , 01, 011, 0101, 01011, 010101

✔ Solução do Exercício 5.2

Escreva uma ER para cada linguagem:

(a) Strings que começam com 0 e terminam com 1:

$$0\Sigma^*1 = 0(0 \cup 1)^*1$$

Note: a menor string aceita é **01**.

(b) Strings com no máximo dois 1s:

$$0^*(1 \cup \varepsilon) 0^*(1 \cup \varepsilon) 0^*$$

Aceita strings com 0, 1 ou 2 ocorrências de 1, com 0s em qualquer posição.

(c) Strings onde todo 0 é seguido de pelo menos um 1:

$$1^*(01^+)^*$$

Começa com 1s opcionais, depois cada 0 deve ser seguido de pelo menos um 1.

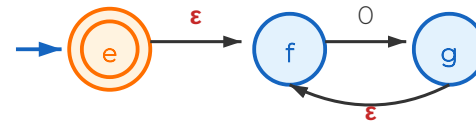
✓ Solução do Exercício 5.3

Construa o NFA para $01 \cup 0^*$

Passo 1: NFA para 01 (concatenação de "0" e "1"):



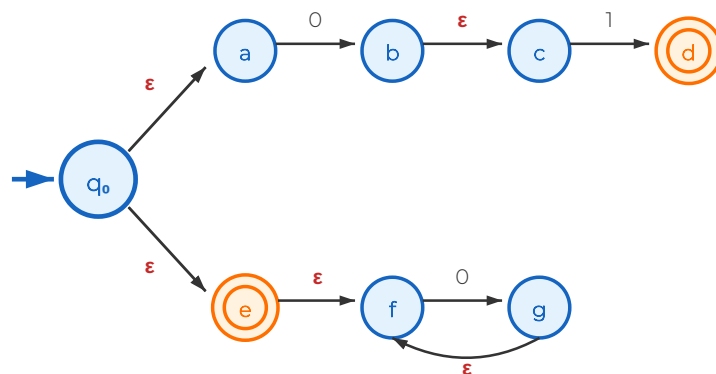
Passo 2: NFA para 0^* (estrela sobre "0"):



✔ Solução do Exercício 5.3 (cont.)

Passo 3: Aplicar a construção de **união** para $01 \cup 0^*$:

Criar um novo estado inicial q_0 com transições- ϵ para o início de cada NFA.



O NFA resultante tem **8 estados** e aceita $L(01 \cup 0^*) = \{01\} \cup \{\epsilon, 0, 00, 000, \dots\}$.

✔ Solução do Exercício 5.4

Simplifique cada ER:

(a) $\emptyset^* \cup 1$

$\emptyset^* \equiv \varepsilon$ (isto é, ambas descrevem a linguagem $\{\varepsilon\}$), portanto:

$$\emptyset^* \cup 1 = \varepsilon \cup 1$$

(b) $R \circ \varepsilon$

ε é a identidade da concatenação, portanto:

$$R \circ \varepsilon = R$$

(c) $R \cup \emptyset$

$\emptyset$ é a identidade da união, portanto:

$$R \cup \emptyset = R$$

✔ Solução do Exercício 5.5

Verdadeiro ou falso?

(a) $L(\emptyset^*) = \{\varepsilon\} \rightarrow \text{Verdadeiro.}$

$\emptyset^*$ significa "zero ou mais concatenações de strings de $\emptyset$ ". Com zero concatenações, obtemos ε . Com uma ou mais, precisaríamos de uma string de $\emptyset$, que não existe. Logo, $L(\emptyset^*) = \{\varepsilon\}$.

(b) $(0^*1^*)^* = (0 \cup 1)^* \rightarrow \text{Verdadeiro.}$

$(\subseteq)$: Toda string de $(0^*1^*)^*$ é uma concatenação de blocos 0^*1^* , ou seja, é uma string de 0s e 1s. $(\supseteq)$: Toda string de $(0 \cup 1)^*$ pode ser escrita como concatenação de blocos, onde cada bloco tem um único símbolo (que é 0^*1^* com repetições 0 ou 1).

(c) $\emptyset \circ R = R \rightarrow \text{Falso.}$

$\emptyset \circ R = \emptyset$ (e não R). Concatenar com o vazio é como multiplicar por zero: o resultado é a linguagem vazia. Para que $xy \in \emptyset \circ R$, precisaríamos de $x \in \emptyset$, o que é impossível.

✔ Solução do Exercício 5.6

Enunciado: $\Sigma = \{a, b, c\}$. Strings que começam e terminam com a e têm pelo menos um b .

Solução:

$$a(a \cup b \cup c)^*b(a \cup b \cup c)^*a$$

Raciocínio:

- Começa com a : primeiro símbolo fixo
- Depois qualquer coisa: $(a \cup b \cup c)^*$
- Pelo menos um b : aparece obrigatoriamente
- Depois qualquer coisa de novo: $(a \cup b \cup c)^*$
- Termina com a : último símbolo fixo

Exemplos: $aba, abca, aabba, abcba$

▮ Note que a menor string aceita é aba (comprimento 3).

✔ Solução do Exercício 5.7

Enunciado: $\Sigma = \{a, b\}$. Strings de tamanho ímpar.

Solução:

$$(a \cup b)((a \cup b)(a \cup b))^*$$

ou, usando a abreviação $\Sigma = (a \cup b)$:

$$\Sigma(\Sigma\Sigma)^*$$

Raciocínio:

- Um símbolo qualquer: Σ (garante tamanho ≥ 1 , ímpar)
- Seguido de zero ou mais **pares** de símbolos: $(\Sigma\Sigma)^*$
- Total: $1 + 2k$ símbolos, que é sempre ímpar

Exemplos: $a, b, aab, bba, ababa$

✔ Solução do Exercício 5.8

Enunciado: Justifique passo a passo que $(01)^* \cup 1(01)^*$ é ER válida.

Passo 1: 0 é ER válida (caso base, símbolo $\in \Sigma$). $L(0) = \{0\}$.

Passo 2: 1 é ER válida (caso base). $L(1) = \{1\}$.

Passo 3: 01 é ER válida (concatenação de Passos 1 e 2). $L(01) = \{01\}$.

Passo 4: $(01)^*$ é ER válida (estrela do Passo 3). $L((01)^*) = \{\epsilon, 01, 0101, \dots\}$.

Passo 5: $1(01)^*$ é ER válida (concatenação dos Passos 2 e 4). $L(1(01)^*) = \{1, 101, 10101, \dots\}$.

Passo 6: $(01)^* \cup 1(01)^*$ é ER válida (união dos Passos 4 e 5).

$$L((01)^* \cup 1(01)^*) = \{\epsilon, 01, 0101, \dots\} \cup \{1, 101, 10101, \dots\}$$

Linguagem: strings da forma $(01)^k$ ou $1(01)^k$ para $k \geq 0$. Equivalentemente, strings que são prefixos da sequência infinita $\dots 010101 \dots$ começando em posição par ou ímpar.